

# DL067 数据并行语义实验

本实验不启动网络、不需要GPU、不配置真实进程组。你会在一个CPU进程中计算多个模型副本的梯度，并检验它们何时等价于完整批目标。完成后应能发现分母、样本覆盖和更新次数错误，而不是只知道分布式API的名字。

## 1 环境与运行

推荐独立Python3.11或3.12。当前实测Python3.12.14、PyTorch2.14.1+cpu，CPU单线程。计算采用FP64，以便清楚区分舍入量级与归约语义错误。没有外部数据或账户，数据由本课固定随机生成器产生。

```
python -m pip install -r requirements.txt
python -m unittest -v test_experiment.py
python -O -m unittest -v test_experiment.py
python experiment.py --output outputs
python make_figures.py
```

在本单元目录运行。默认会覆盖同名结果；修改实验时用--output另存。14项测试在普通与-O模式都应实际执行，通过不依赖会被优化删除的assert语句。运行结果不会包含“多GPU已验证”，因为没有做这项工作。

## 2 纸笔先定义目标

设rank0有20例、rank1有30例、rank2有78例，各自算平均loss。写出全局128例平均loss对应的权重。再假设真正DDP最后平均三个梯度，推导局部loss应乘的修正系数。

不要直接复制实验函数到DDP后再保持同样除法。模拟average\_gradients直接算最终加权结果，DDP还会执行自己的归约。正确迁移依赖你知道每层归一化到底发生了几次。

## 3 读一次模拟更新

simulate\_step对每个rank复制同一模型，在对应数据子集上计算局部均值梯度。所有副本都从同一个更新前参数点出发，得到全部梯度后才组合，最后主参考模型的优化器更新一次。

```
import copy
import torch
import experiment as e
x, y = e.data()
base = e.model()
parts = [torch.arange(0,20), torch.arange(20,50), torch.arange(50,128)]
rows = [e.gradient(copy.deepcopy(base), x[p], y[p]) for p in parts]
g = e.average_gradients([v[0] for v in rows], [v[1] for v in rows])
full = e.gradient(copy.deepcopy(base), x, y)[0]
print((e.flatten(g) - e.flatten(full)).abs().max())
```

本次最大差约 $8.33\text{e-}17$ 。再把mode改为rank\_mean，检查偏差明显增大。这个变化只修改权重，不改模型或数据，因此能定位问题来源。

## 4 有效计数与零贡献rank

掩码实验把128例分成两个64行rank，但只有前一个64例与后一个7例有效。目标按71个有效例子平均。读取masked\_gradient\_max\_errors，正确加权约 $1.11\text{e-}16$ ，错误rank均值约0.07391。

gradient允许局部mask全0，返回零梯度和计数0。真实系统中这类rank仍可能必须参与集体通信，不能悄悄退出。average\_gradients拒绝全局计数为0，因为此时目标均值没有定义。

**故障植入。** 在自己的副本中把有效计数换成本地行数。即使每rank行数相同，掩码测试也会失败。用一句话解释这与变长序列token loss的联系。

## 5 实际检查采样器索引

本课调用真正的DistributedSampler生成索引，但不启动分布式训练。运行sampler\_indices，确认N=10、world=3的两种结果。填充时共12个条目但只有10个不同ID，0和1重复；丢弃时共9个不同ID，9不出现。

```
import experiment as e
for drop in [False, True]:
    parts = e.sampler_indices(drop_last=drop)
    flat = sum(parts, [])
    print(drop, parts, len(flat), len(set(flat)))
print(e.sampler_indices(shuffle=True, epoch=0))
print(e.sampler_indices(shuffle=True, epoch=1))
```

解释训练与评估为什么对重复条目可能有不同容忍度。对全局准确率，归约“正确个数”和“有效总数”后相除；各rank样本数不等时，简单平均准确率通常错误。若填充重复测试ID，还应去重或设计无填充评估划分。

## 6 三种子的真正训练

每个初始化种子11、23、37训练四种模式80步：完整批、正确加权、错误rank均值、重复最前20例。模型5到12到2，Tanh，SGD学习率0.15、动量0.8。完整批与正确加权使用相同样本和目标，最后参数最大差约 $6\text{e-}16$ 以内。

查看history中的完整批与加权测试loss是否逐点一致。再看另外两个模式为何不同。错误rank均值没有丢掉样本，却改变样本权重；重复rank没有增加新数据，只把同一20例重复贡献三次。

12个pt文件包含真实最终权重、种子、模式与步数。用weights\_only=True读取可信文件，按同样FP64结构加载，并在data.npz的测试集上复核准确率与交叉熵。它们用于终点复核，不是完整中途续训包，因为未包含优化器与随机状态。

## 7 通信图的正确使用

ring\_model只计算公式，不发出任何网络通信。输入100 MiB、world=4、每段5微秒、12.5 GB/s，应得到约12.612912毫秒。解释每个单位，把MiB先换成字节。

这个估计不包含拓扑、拥塞、bucket选择与计算重叠。不能把它除以CPU脚本运行时间来得到“分布式加速比”。如果要做真实通信测量，必须另有真实进程组、硬件和同步协议，并记录失败与异常。

## 8 Notebook与重建

Notebook在新Python进程的真实IPython进程内内核顺序执行，包含单步梯度、采样器、14项测试以及全部三种子四模式重训。重跑结果写入notebook\_outputs，图像以PNG输出内嵌。不声称检验浏览器Jupyter界面，也不声称测试多进程通信。

```
python create_notebook.py
python execute_notebook.py experiment.ipynb
python build_pdf.py lecture.md
python build_pdf.py lab.md
python build_pdf.py answers.md
```

## 9 练习

- 两rank分别有2和6个例子，局部平均梯度为1和3。求全局梯度；若DDP做平均，局部loss应分别乘几倍？
- 两rank各4例，局部平均梯度2和6。DDP平均后又除world size，结果是什么？
- 四rank、本地微批16、累积3次，有效处理行数多少？如果每个rank最后一个微批只有8行，实际多少？
- $N=10$ 、 $R=3$ 且不打乱，列出填充与丢弃索引。说明测试集直接沿用填充会出现什么问题。
- 写出一个不会被本课模拟发现的真实分布式错误，并设计检测方法。
- 有rank准确率9/10，另一rank50/100。求全局准确率与错误等rank平均。
- 解释数据并行、参数分片和张量并行各自分的是什么。

提交目标推导、测试结果、采样ID审计、checkpoint复核和一页结论。结论必须同时写出已验证的数学语义与尚未验证的通信系统行为。